Memory and Loop Design in a Sandboxed ReAct Agent Harness

Eyouale Tensae Degefaw
Fakultät für Informatik und Mathematik
University of Passau
Stuttgart, Germany
degefa01@ads.uni-passau.de

Abstract*—Large Language Models (LLMs) are inherently stateless across invocations, degrade in performance as their context window fills, and can execute destructive actions on host systems when granted tool access. Existing agent frameworks typically address these challenges in isolation, leaving the interplay between loop control, memory management, and execution safety under-specified. This paper presents *Nikola*, a sandboxed ReAct agent harness integrating multi-tier memory, Model Context Protocol (MCP) tooling, declarative permissions, and parallel sub-agents. The ReAct loop terminates via a structural finish signal or an iteration cap guard; context token limits trigger progressive compaction rather than termination, while tool failures are surfaced directly as observations. To maintain context budgets, the compaction pipeline truncates bulky tool outputs and evicts non-critical turns, then summarizes the evicted turns into a recap promoted to the persistent store. Persistent memory combines a BM25-ranked SQLite FTS5 index for exact recall with Qdrant dense embeddings for semantic retrieval. Execution safety is enforced through a first-match-wins declarative policy (`allow`, `ask`, `deny`); interactive confirmation prompts degrade automatically to `deny` within non-interactive sub-agents to prevent deadlocks. Sub-agents execute concurrently in isolation and return structured summaries, preventing parent context inflation. Comprehensive observability is provided via Prometheus, Loki, and Grafana to track turn latency, token usage, and tool invocations. This report describes two design outcomes: (1) using SQLite as the sole source of truth with a disposable vector index enables network-independent cross-session persistence, and (2) bypassing chain-of-thought generation during turn summarization reduces completion overhead from 1,016 to 81 tokens on a single manually selected six-turn transcript, with no degradation apparent on inspection of that transcript.

***Index Terms*—agent observability, context compaction, declarative permissions, Model Context Protocol, ReAct harness, retrieval-augmented memory, sandboxed tool execution.**

I. Introduction

Large language models are stateless: each invocation maps a prompt to a continuation with no memory of prior calls. Agentic systems obtain goal-directed behavior by placing such a model in a loop in which it reasons about the task, acts by invoking a tool, and observes the result before reasoning again [1]. The model contributes only the reasoning step; the surrounding infrastructure loop control, tools, permission checks, and the state carried between steps is what we call the *harness*.

Statelessness leaves the harness two problems. Because every step appends observations to the prompt, the context window fills as a run progresses and the model's effective use of that context degrades with it, a phenomenon termed *context rot* [2]. And when a run terminates its context is discarded, so nothing learned in one run is available to the next.

Nikola is a containerized ReAct harness we built in the AI Engineering Lab at the University of Passau (summer semester 2026). Its loop is bounded by an iteration cap; tools are discovered at runtime through an MCP client over stdio and HTTP and governed by declarative `allow`/`ask`/`deny` permissions; work can be delegated to isolated sub-agents, including in parallel; and the system is instrumented with Prometheus, Grafana, Mitmproxy and Loki.

Memory handling is the focus of this report. On the short-term side we manage the context window with a compaction pipeline that truncates tool outputs and drops non-critical turns, and summarizes what it drops into a recap that outlives the window; on the long-term side we combine SQLite with Full-Text Search 5 (FTS5) as the source of truth with Qdrant embeddings for semantic retrieval.

The remainder is organized as follows. Section II gives a system overview, Section III the agent loop, and Section IV, the core of the report, memory handling. Section V reports how we went beyond the minimum requirements, Section VI the extensions we did not pursue and why, and Section VII concludes.

II. System Overview

Nikola is a command-line agent that runs inside a container; a single run command opens an interactive session.

The interface is a Typer-based read-eval-print loop (REPL) with twelve slash commands, tab completion on any unambiguous prefix, and command history. Six inspect or steer the session (`/help`, `/mode`, `/model`, `/tools`, `/clear`, `/exit`) and six expose the subsystems this report describes: `/context` reports budget utilization, `/compact` condenses the conversation by hand (Section IV-F), `/rules` and `/skill` list the just-in-time context of Section IV-D, `/permissions` shows how the gate resolved recent calls, and `/mcp` lists discovered servers. Model, temperature, and loop limits are declared in `config.yaml`, while the access token is supplied only as an environment variable and is never committed. Requests reach the InnKube endpoint through a three-part LLM layer: a client factory, typed protocols, and streaming

and non-streaming dispatchers. Above it sits the bounded ReAct loop of Section III.

TABLE I
NIKOLA COMPONENTS AND THEIR KEY PROPERTIES

Layer	Component	Key properties
Interface	Typer REPL	Slash commands, tab completion, history
Config	`config.yaml`	Model, temperature, loop limits; token via env var only
LLM access	Three-part LLM layer	Factory, typed protocols, dispatchers; InnKube
Control	ReAct loop	Ends when the model returns no `tool_calls`, or at the iteration cap (default 100); the token budget compacts rather than terminates (Section III)
Dispatch	ToolRegistry	`dispatch(name, args)` → `string`; "`Error: ...`"
Tools	Sixteen built in	Ten filesystem, `execute_command`, `load_skill`, two memory, two delegation; none deletes a file
Shell	`execute_command`	Word-level screen for deletion, escalation, host control, device writes; capped output; secrets stripped; guardrail, not proof
MCP	stdio, HTTP/SSE	Runtime discovery; custom Calendar tool
Memory	FTS5 + Qdrant	SQLite sole source of truth, BM25 recall; Qdrant a disposable accelerator (Section IV)
Policy	`permissions.yaml`	`allow` / `ask` / `deny`; glob on tool name and optional path; first match wins, then built-in lists
Delegation	Sub-agents	`file-scout`, `scheduler`, `general`; `delegate` and `delegate_parallel`
Observability	Lifecycle hooks	Prometheus, Grafana, Alloy → Loki
Isolation	`resolve_safe_path()`	Resolve-then-contain check; plus non-root container, local-only ports
Validation	pytest	524 tests (467 unit, 30 integration, 27 end-to-end), including memory persistence and sub-agent delegation; two unit tests assert a superseded iteration default

Every tool call is screened before it is dispatched. The call is matched against `permissions.yaml`, whose rules pair a glob on the tool name with an optional second glob on the `path` argument; the first matching rule wins, and a call matching no rule falls through to built-in allow, ask, and deny lists and then to a configured default.

Tools are exposed through a registry with a uniform `dispatch(name, args)` → `string` signature, with failures returned as "`Error: ...`" strings. Sixteen tools are built in: ten filesystem operations, `execute_command`, `load_skill`, the `remember` and `search_memory` pair of Section IV, and the two delegation tools below. No tool in the registry deletes a file. MCP clients over stdio and HTTP/SSE discover further tools at runtime and register them into the same registry, among them our custom Calendar tool.

Shell access is where the no-deletion property is easiest to lose, so `execute_command` enforces it rather than trusting the prompt. The command line is lexed into shell words, operators included, so every segment of a pipeline is screened and not merely the first; substitution punctuation is blanked beforehand so that `$(rm x)` and backticks cannot smuggle a word past the check. Any word whose basename names a deletion utility, a privilege-escalation command, a host or process control, or a device-level write is refused, as is a deletion flag such as `--delete` on an otherwise harmless

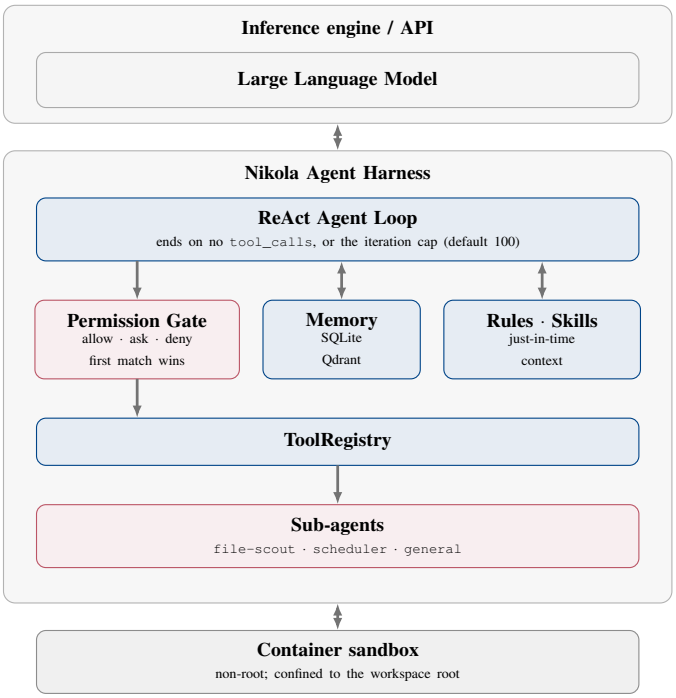


Fig. 1. Nikola's harness layers. Every tool call passes the permission gate before it reaches the registry. The loop itself is a state machine, given in Fig. 2.

command. Quoting is honored, so a quoted string remains data. Each run is bounded by a timeout that kills the whole process group, its output is capped per stream, and variables whose names look like secrets are stripped from the child environment so that `env` cannot print the access token into the transcript. We are explicit that this is a guardrail and not a proof: a deletion hidden inside a script the agent writes and then executes cannot be caught by inspecting a command line, which is why the container, not the screen, remains the security perimeter.

Work may be delegated to isolated sub-agents `file-scout`, `scheduler`, and `general` each with its own prompt, tool set, and budget, through a `delegate` tool and a `delegate_parallel` variant that gathers several concurrently. The schema advises the model to submit at most five at a time, but that ceiling is advice in the tool description rather than a check in the code. Long-term memory combines SQLite FTS5 with a Qdrant service and is treated in Section IV. Lifecycle hooks feed Prometheus at `:9464/metrics` behind a Grafana dashboard, and structured JSON logs travel via Alloy to Loki.

Table I summarizes the components, and Fig. 1 shows how they connect.

III. THE AGENT LOOP

A. Execution Model and State Machine

We organize execution into three nested scopes: a Session spans the application lifecycle, a Turn spans one user input until the final answer, and a Step is one ReAct iteration: a single LLM call together with any tool calls it requests.

The loop is produced by a closure factory, `make_process_turn_fn_v2()`, which receives the dispatcher, tool registry, configuration, and output sink by injection and returns an asynchronous coroutine function `async (messages) -> str`. Injection rather than module-level state keeps globals and I/O out of the loop, so the test suite exercises the real state machine against a mocked LLM.

A Turn occupies one of two primary states: *Reasoning*, in which the model is called, and *Acting*, in which tools are executed, realizing the reason–act–observe cycle of ReAct [1]; Fig. 2 summarizes the resulting state machine. The transition is structural rather than textual: if the assistant message carries `tool_calls`, the loop enters Acting, dispatches each call through the ToolRegistry, appends each result as a `tool` message, and returns to Reasoning; otherwise, the text content is treated as the final answer. We do not parse "`Thought:`" or "`Action:`" prefixes: the model's formatting can drift without notice, whereas the structural `tool_calls` field is part of the API contract. Tool schemas are fetched dynamically from the registry on every iteration, enabling tools discovered at runtime over MCP to reach the model without modifying loop code.

B. Guard Conditions and Termination

The loop is guaranteed to terminate, governed by one active mechanism: the iteration cap in `loop.max_iterations` (defaulting to 100). Three conditions commonly associated with termination do not end a Turn:

1) **Token budget**: Crossing the context token budget triggers sliding-window compaction rather than termination (Section IV).
2) **Tool failures**: Tool exceptions become observation strings returned to the model rather than fatal loop exits (Section III-C).
3) **Wall-clock limit**: The engine's `TurnLimits` structure carries a `max_time_seconds` field, but it defaults to `None`, is omitted from `config.yaml`, and no caller sets it; the per-request `llm.timeout_seconds` bounds one HTTP request, not the cumulative loop. We report this inert time budget as an architectural gap rather than a design choice.

At the cap the engine raises nothing: it returns a synthetic final answer, "`[Loop terminated: max iterations (100) reached]`", and appends it to the history as an assistant message. Every caller (the REPL, automated tests, and sub-agents) therefore keeps the single-string return contract and a history valid for subsequent turns.

Two further edge cases follow the same principle: empty `choices` from the provider and unhandled network exceptions produce bracketed error strings rather than crashing the harness. A `finish_reason` of `"length"` is a third, captured by the streaming adapter but not intercepted by the engine, so truncated completions enter history without a diagnostic tag.

C. Error Handling and Streaming

Failures are treated as observations rather than exceptions. Malformed JSON payloads, non-object arguments, unknown tool names, and tool-internal exceptions are intercepted and returned to the model as "`Error: ...`" observations, so it can inspect the failure and retry.

Streaming is coordinated by `accumulate_stream()`, which rebuilds one assistant message from the chunk sequence. Text deltas reach the output sinks in real time, while tool-call deltas are indexed and concatenated by `id`, `name`, and arguments (with a synthetic `call_{index}` identifier when the provider omits one). The loop therefore makes exactly one LLM round-trip per iteration in both modes. Reasoning models such as the Qwen-3 series stream chain of thought in a separate `reasoning_content` field; it is displayed live but never persisted to the history, which would pollute the next call.

D. Integration Points

Four decoupled integration points interface with the loop:

- **Permission Gate**: Evaluates each tool call prior to execution and returns `allow`, `ask`, or `deny`.
- **Lifecycle Hooks**: Nine non-intrusive hooks carry the observability layer: `on_run_start` and `on_run_end` bound a run, `on_llm_call` and `on_tool_call` track Step work, three `on_subagent_*` hooks (start, progress, end) follow delegation, and `on_memory_operation` and `on_mcp_server` observe the store and MCP lifecycle. Permission decisions travel inside the tool-call event.
- **Compaction**: Intercepts turns when history token counts exceed the configured budget.
- **Memory Retrieval**: Pre-turn retrieval runs in the REPL caller, just before the turn coroutine. Because it sits there rather than in the loop core, sub-agents, which reuse the engine with their own limits and tools, get no automated long-term retrieval.

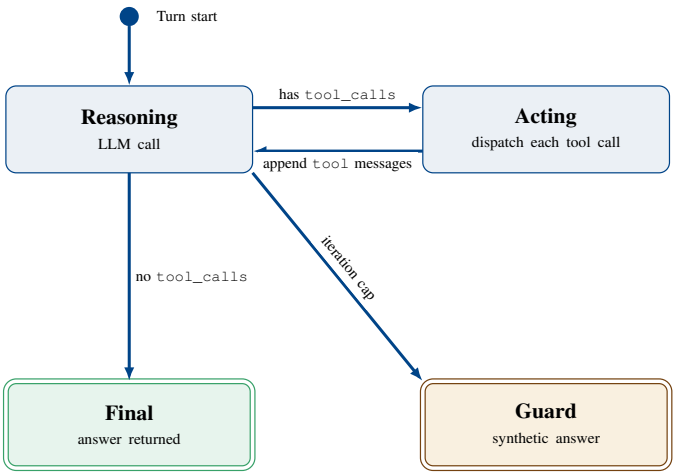


Fig. 2. The Turn loop as a state machine. Reasoning moves to Acting while the model returns `tool_calls`, and to Final when it does not. The iteration cap is the only guard that terminates the loop: it enters Guard, which writes a synthetic answer to history rather than raising. Transitions test the structural `tool_calls` field, not parsed text.

```python
# In REPL, before process_turn is called:
prompt = retrieve_memories(prompt) + prompt

async def process_turn(messages):
    for step in range(config.loop.max_iterations):
        if over_token_budget(messages):
            # Context compaction; does not terminate turn
            messages = compact(messages)

        # Reasoning
        schemas = registry.schemas()
        reply = await dispatch(messages, schemas)
        if not reply.choices:
            return error("no response from LLM")

        messages.append(reply)
        if not reply.tool_calls:
            return reply.content        # Final Answer

        # Acting (sequential dispatch prevents confirmation races)
        for call in reply.tool_calls:
            if allowed(call):
                result = await registry.dispatch(call)
            else:
                result = denied(call)
            messages.append(tool_message(result))

    # Iteration cap reached
    return guard("max iterations reached")
```

Listing 1. The Turn Loop (simplified representation; dispatcher, registry, and limits injected).

IV. Memory Handling

A. Scope and Failure Modes

We divide memory by how long it lives, not by what it holds. Short-term memory is the context window of one Session (system prompt, pinned rules, tool schemas, conversation, observations), gone when the process exits unless the write policy of Section IV-F has put something in the store. Long-term memory is that store, a database on disk surviving Session, restart and container. A third tier, rules and skills on disk, sits outside it (Section IV-D).

The two fail differently: the window by filling, the store by being empty. Each gets its own policy, and they meet only in Sections IV-C and IV-F.

B. Short-Term Memory: Structure and Validity

The harness rebuilds and sends the whole conversation on every call, holding it as an ordered list of typed message dictionaries,

$$M = [m_0, m_1, m_2, \ldots, m_n], \quad m_i \in \texttt{ChatCompletionMessageParam}, \tag{1}$$

owned by the Session and laid out in Table II. Only the history and the observations grow.

OpenAI-compatible endpoints impose one structural constraint: an assistant message carrying `tool_calls` must be followed immediately by its matching `tool` observations, or the request returns HTTP 400. We therefore treat the user message, the assistant's tool-calling message and its observations as one indivisible Turn block,

$$T = \langle u, a, o_1, \ldots, o_k \rangle, \tag{2}$$

kept or dropped whole. Finer granularity would risk an invalid history rather than merely a worse answer.

TABLE II
The Short-Term Memory Message Array

Index	Role	Content or payload
m_0	`system`	Agent prompt, pinned rules, and pinned memories
m_1	`user`	User prompt, with the top three retrieved memories prepended
m_2	`assistant`	Model response, optionally carrying `tool_calls`
m_3	`tool`	Execution observation, matching the preceding `tool_call_id`
m_4	`user`	The next user Turn, and so on

m_0 is fixed for the Session. Messages m_1–m_3 form one Turn block (eq. 2) and are retained or evicted together.

C. Context Degradation and the Compaction Cascade

Quality degrades before the window fills [2], so we estimate buffer size at 3.5 characters per token. That is a heuristic, since InnKube exposes no tokenizer here, and one that may underfire on token-dense inputs. Compaction fires on the budget stated with Table III, checked at every Step.

A three-tier cascade then runs cheapest first, exiting as soon as the context fits. Truncation goes first because one oversized file read can fill most of the buffer, freeing the most budget without losing dialogue; eviction follows, preserving (2) by construction. Tier 3 then summarizes what tier 2 dropped, and that recap is promoted to the store (Section IV-F).

A recap already in the buffer is folded into the new one rather than stacked, so the conversation carries one evolving summary rather than a pile.

One limitation is architectural. The trigger estimate excludes tool schemas while the `/context` estimator counts them, so compaction fires later than reported utilization suggests: the adapter protocol defines its estimator over messages alone, and the budget check runs before schemas are fetched. Closing it is a protocol change, and we treat it as future work.

Subordinate LLM calls run with `reasoning_effort: none` and a capped budget. The models served here emit chain-of-thought before any visible answer token; on a single manually selected six-turn transcript, disabling reasoning reduced completion cost from 1,016 tokens to 81 with no degradation apparent on inspection of that transcript. The right setting for a subordinate call is not the right setting for the main loop.

D. Static Window Contents: Rules and Skills

Not everything in the window comes from the conversation. Rules are always-active instructions injected into the system prompt; skills are the opposite, procedural knowledge held on disk and loaded only when the `load_skill` tool asks, because such documentation is large and seldom needed and loading it up front would waste budget in every Session that never invokes it. Section V-B gives both paths.

TABLE III
THE THREE-TIER COMPACTION CASCADE

Tier	Mechanism and trigger	Cost	Information preserved
1	Observation truncation, when any single observation exceeds `max_observation_chars` (10,000)	$O(1)$ slicing; no LLM call	Head and tail boundaries, plus a "`[truncated]`" marker
2	Atomic Turn-block eviction, when the total still exceeds the budget after tier 1	$O(k)$ slicing; no LLM call	Pairing invariant; the system message and latest user message are protected
3	Abstractive summarization, when tier 2 has evicted anything	One capped LLM completion	A semantic recap of the evicted Turns, inserted at position 1

Entered when the estimated context exceeds `window_tokens` − `reserve_output_tokens` = 253,952 tokens, checked at every Step, and exited as soon as the context fits. The tier-3 cap is `summary_max_tokens` = 1,500 completion tokens, and its recap is promoted to the store (Section IV-F).

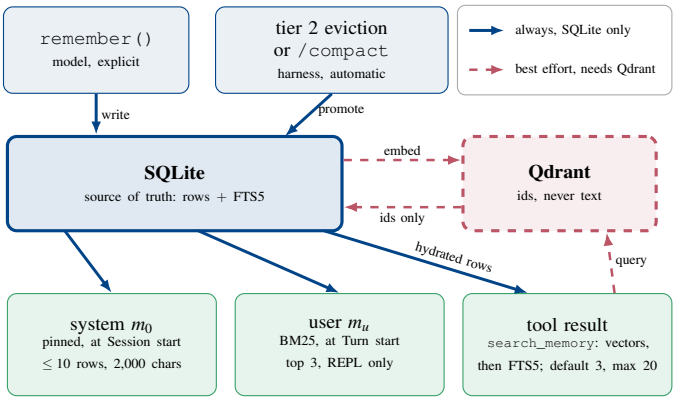


Fig. 3. Long-term memory end to end: who writes, where a row lands, and which part of the window a read reaches. Qdrant holds identifiers, never text, so every byte comes from SQLite an outage costs recall quality, never correctness; the `embedded` flag has the collection backfilled at the next start.

E. How Long-Term Memory Is Stored

A memory on disk costs tokens only on the Turns that read it. Two engines hold it (Fig. 3): SQLite owns the rows in `.nikola/.memory/memories.db` behind an FTS5 index for BM25 word queries, and Qdrant holds `octen-embedding-8b` embeddings for meaning queries by cosine distance.

The two are not equals by design. A write reaches SQLite immediately and Qdrant only if it can: SQLite is always safe but matches words, not meaning, while Qdrant finds meaning and loses data if the daemon dies. The split confines any failure to search quality alone.

Rows are typed (`agent/memory/store.py`): `kind` separates a checkable `fact` from a `preference` and a `summary` recap, `source` records whether the model or the machine wrote it so the agent can trace why it believes something, and `pinned` marks rows paid for on every Turn. `created_at` is written but never read: ranking is by match alone, so an old recap competes with a new one.

Triggers keep the FTS5 index in step rather than application code: a process dying between the row write and the index update leaves the two inconsistent with nothing to undo it, while a trigger runs inside the same SQLite transaction, so both land or both fail.

The current design assumes a trusted single-user environment: the memory store has no write authentication, so any process with filesystem access to `memories.db` can insert rows. It is not hardened against adversarial or multi-tenant use.

F. Memory Policy: Write and Read

Two paths write, bounded as Table IV states: the model calls `remember` explicitly, or the harness promotes a recap under the current Session id when tier 2 evicts Turns, so forgetting detail in the window still leaves something durable. Promotion overwrites rather than appends, so a long conversation keeps one growing summary, and `/compact` uses the same path by hand.

Every write passes one guard module, rejecting empty, whitespace-only and `Error:`-prefixed content on both paths (Listing 2). The last check matters because a failed tool returns an observation starting with that prefix rather than raising, so without it the agent would file its own failures as facts. Credentials, ephemeral state and raw tool output are excluded by convention, not by check. Two packaged rule files advise the model on what is worth storing, loaded like all other workspace rules (Section IV-D).

Reading happens at three moments. At Session start the harness puts pinned rows into the system message, oldest first; that text stays for the whole Session, so the bound protects working context, and because no row can be unpinned, once more than ten are pinned the oldest keep the places. At Turn start it prepends the closest rows by BM25 without touching Qdrant even when it is running, so this path is always on and fails open, any error swallowed and the Turn proceeding with the plain prompt. This path is the REPL's alone; sub-agent Turns get neither it nor the pinned rows. The third is the model's own: `search_memory` queries the vector store first, then fills from BM25. It is the only path touching Qdrant, so an outage costs its results and nothing else.

Three limits apply to both directions. No unpin, delete or update tool exists, so pinned rows accumulate and a changed fact leaves a stale row beside the correct one. Writes always insert, so a fact remembered twice takes two slots of every top-N. Replacement is per Session, so earlier summaries pile up.

```
def should_store_content(content):
    if not content:
        return False, "Content is empty"
    if not content.strip():
        return False, "Content is whitespace only"
    if content.startswith("Error:"):
        return False, "Content is an error string"
    return True, ""
```

Listing 2. The write guard (`agent/memory/policy.py`).

TABLE IV
MEMORY POLICY: WRITE PATHS AND READ STAGES

Path / Stage	Trigger or content	Source / Target	Bound
Explicit write	Model calls `remember`	`user`	Guards apply
Auto-promote	Tier 2 eviction or `/compact`	`compaction`	Replaces prior summary for the Session
Rejected	Empty, whitespace-only, or `Error:`-prefixed	—	Enforced on both paths
Convention	Credentials, ephemeral state, raw output	—	No check
Bootstrap read	Pinned rows at Session start	system m_0	$\leq$10 rows, oldest first; 2,000 chars beyond first
Per-Turn read	BM25, before user message	user m_u	Top 3; fails open; REPL only
On-demand read	`search_memory`, vectors + BM25	tool result	Default 3, max 20

TABLE V
SELF-DIRECTED EXTENSIONS

Area	What we added
Context window	Compaction cascade, character-per-token budget estimate, reasoning suppression (1,016 to 81 tokens)
Instruction layers	Always-on rules; 5 skills behind `load_skill`, a workspace skill shadowing a built-in
Workspace surface	The `.nikola/` contract: rules, skills, MCP, logs, store (Table VI)
Long-term memory	Qdrant vectors, hybrid search, `embedded` backfill after an outage
MCP	Both transports in use at once; calendar server with 5 tools
Filesystem tools	Four past the required capabilities: `glob`, `grep`, `search_and_replace`, `update_todo_list`
Shell access	`execute_command`: 27 denied executables, 4 denied flags, pinned cwd, 5,000-char streams, scrubbed env
Permissions	Path-scoped rules, first match wins, layered files, tiered built-in defaults, an argument-aware decision cache that `ask` bypasses by design
Sub-agents	`delegate_parallel` over `asyncio.gather`, 3 roles, live per-agent tool and token counters, the parent's loop reused
Terminal	Streaming and non-streaming dispatch, `--thinking`, `--doctor`, Tab completion, and introspection commands: `/context`, `/rules`, `/skill`, `/mcp`
Observability	19 metric families, 31 panels, mitmproxy HTTP Archive (HAR) capture, SQLite-web
Failure behaviour	Qdrant, Prometheus and Loki each soft-fail: an outage costs search quality or telemetry, never a Turn
Tests	467 unit, 30 integration, and 27 end-to-end, all in CI; two unit tests assert a superseded iteration default and are retained as regression markers
Documentation	26-page site on GitLab Pages, 3 Architecture Decision Records (ADRs) (Table VII)

V. BEYOND THE MINIMUM REQUIREMENTS

The lab specification fixes a minimum for each week (the ReAct loop, a tool registry and a container; MCP integration and a persistent store; then observability, permissions and sub-agents) and states that meeting it earns a passing grade at most, with anything above resting on extensions the group designs itself. The preceding sections do not separate the two; this one does. Table V lists what we added; the prose reports what it cost.

TABLE VI
THE WORKSPACE EXTENSION SURFACE

Path in the workspace	What it holds, and the rule that governs it
`.nikola/rules/*.md`	Always-on instructions appended to the system prompt; workspace files plus two packaged defaults
`.nikola/skills/`	On-demand bundles, `*.md` or `*/SKILL.md`; a workspace skill shadows a built-in of the same name
`.nikola/mcp.json`	Server declarations with environment substitution; calendar over HTTP, `chrome-devtools` over stdio
`.nikola/logs/agent.jsonl`	One JSON object per record; rotates at 5 MB keeping 3, tailed into Loki by Alloy
`.nikola/.memory/memories.db`	SQLite rows and the FTS5 index; the only state that outlives the Session
`permissions.yaml`	Permission rules; this copy outranks the project-root one

TABLE VII
PUBLISHED DOCUMENTATION SITE NAVIGATION

Group	Pages	Contents
Home	1	Summary, service table, quick start
Getting Started	3	Installation; configuration reference; services and endpoints
Concepts	9	Execution model; tools and registry; workspace sandbox; skills; rules; MCP; long-term memory; permissions; sub-agents
Architecture	2	System overview; observability stack
Use Cases	4	File search; calendar scheduling; parallel sub-agents; custom MCP tool
Demo	1	Seven-scenario walkthrough
Testing	1	Suites, CI stages, the two spec E2E cases
Decisions	4	ADR index; permission rules; sub-agent auto-deny; memory split
About	1	Project and naming
Total	**26**	Built by `mkdocs build --strict`, deployed by the `pages` CI job

A. Context Engineering and Hybrid Retrieval

No week's minimum mentions the context window, so all of Sections IV-C–IV-D is self-directed: the compaction cascade, the estimate that triggers it, and the suppression of chain-of-thought during summarization. The cost is a code path that can corrupt the window it maintains—dropping Turns violates the pairing invariant of Section IV-B unless each drop is atomic, which is why compaction and not the loop owns the operation.

For long-term memory the minimum is a single guarantee: store in one Session, read it back in the next after a restart, which SQLite and FTS5 satisfy alone. Qdrant, the hybrid on-demand path, and the `embedded` backfill flag all sit beyond it, buying semantic recall at the price of a second engine to operate; Section IV-E argues the split that makes it disposable.

B. The Workspace Contract and the Terminal

Everything an operator can change without touching the package lives in one tree inside the workspace, and no week requires any of it (Table VI). Rules there stay in the window for the whole Session; skills are names until `load_skill` pulls a body in, and a workspace skill shadows a built-in of the same name. The three mounted here track parcels, read a canteen menu and advise on clothing, none of them harness concerns. `.nikola/mcp.json` exercises both transports at

once: the calendar server over HTTP, `chrome-devtools` over stdio.

The terminal goes past the required basic functionality, and both dispatch modes are in use: streaming for the REPL, non-streaming for the programmatic callers (sub-agents and tests, Section III). Chain-of-thought is displayed under `--thinking` and never written back into the history. Slash commands are declared by decorator, so `/help` and Tab completion come from one registry, and `--doctor` reports why completion is or is not working.

C. Execution Safety and Delegation

The permission requirement is three outcomes: allow, deny, and require-confirmation. We added argument-level scope: a rule may carry a `path_pattern` glob matched against the call's path argument, so `write_file` on `.env*` is denied while `write_file` elsewhere only asks; resolution is first match wins over layered files. Decisions are cached per Session and keyed on the arguments, so approving one path approves only that path, and `ask` bypasses the cache entirely because require-user-confirmation means every call. Delegation goes past the required delegate-and-return: `delegate_parallel` runs sub-agents concurrently through `asyncio.gather`, each in one of three roles with its own iteration cap, allowlist and permissions, and each running the parent's loop rather than a second implementation of it.

`execute_command` is wholly self-directed. Handing a shell to the model would void two harness guarantees at once, so every command is screened before it runs (Section II, Table V). A screen over a command line is a guardrail and not a proof, since a deletion hidden inside `python3 wipe.py` passes it, so the shipped `permissions.yaml` denies the tool outright. That we built it and then switched it off is an honest measure of what screening a command line is worth.

D. Testing, Inspection, and Documentation

The specification asks for unit and end-to-end tests. We added a tier between them, because neither can see wiring: a unit test proves a function works and an end-to-end test needs a live endpoint, while the integration tests run the real loop against a mocked model to assert a component is reachable from it. `test_loop_integration.py` does that for MCP servers, `load_skill` and workspace rules. All three tiers run in CI.

Two inspection surfaces no requirement asks for sit beside the required dashboard: mitmproxy records streaming LLM traffic to HAR files and SQLite-web exposes the memory store mid-Session, at a price of twelve Compose services on `127.0.0.1`. Nothing asks for documentation either; we publish the 26 pages of Table VII through GitLab Pages,[1] including three decision records for choices the code cannot explain: first-match-wins resolution, sub-agent auto-deny, and the split of Section IV-E.

[1]Documentation: https://group-3-c8095f.pages.fim.uni-passau.de/. Source: https://git.fim.uni-passau.de/aie/ai-engineering-lab/student-projects-summer-2026/group-3.

VI. Extensions Not Pursued

Three planned extensions are absent, each for a distinct reason.

The first is a measurement, not a missing feature. Both dispatchers exist (Section V-B), so the benchmark is obvious: does streaming alter time-to-first-token, total latency, or completion cost? We did not run it. A defensible answer needs a fixed task set, enough repetitions to separate the effect from the variance of a shared, multi-tenant endpoint under uncontrolled load, and precise token accounting the endpoint does not expose, the same omission behind the character-per-token heuristic of Section IV-C. The one measurement we do report was affordable because it avoided all three: a single call, one variable changed, an effect far larger than the noise floor. We prefer no measurement to one we cannot defend.

The second is a testing strategy we started and withdrew. HTTP Archive (HAR)-based recording and replay was meant to drive deterministic integration tests with no live LLM endpoint; it worked in prototype, but holding it reliable across streaming responses and an evolving tool-calling protocol needed custom replay adapters and a restructured test harness. A brittle mock layer was not worth the remaining time, so the HAR-replay suite came out in favor of isolated unit tests and live end-to-end verification against the container workspace.

The third is scope we did not reach. Session persistence (serializing and reloading conversation history across invocations) would have overlapped the store's cross-session role, and refining the store was the better use of the time; per-turn mode and model switching stayed a stretch goal. The store still lacks explicit unpin, delete, or update tools, which Section IV-F identifies as an accumulating cost rather than an intentional design constraint, and they are the first thing we would build next.

VII. Conclusion

Nikola's interesting properties are all in what surrounds the model. Statelessness is answered in two places at once, by a compaction cascade keeping one Session inside its window and a store that outlives it, rather than by one mechanism asked to do both jobs. Making SQLite the sole source of truth and the vector index disposable buys semantic recall without making persistence depend on a service that can be down. Bounding the loop on a structural signal rather than parsed text, and screening tool calls declaratively rather than by instruction, keeps both properties checkable by reading one file each.

What we would change first is not a missing feature but a habit. Every gap this report names (an estimate excluding tool schemas, a store with no delete, a screened shell shipped switched off) was found by reading the code against its own claims.

Acknowledgment

The author thanks Prof. Dr. Steffen Herbold for organizing this lab, and teaching assistants Lukas Schulte and Fabian Camilo Peña Lozano for their guidance and support throughout the course.

Parts of this work were assisted by generative AI. We used models from Anthropic's Claude family, principally Claude Opus 5, and Microsoft Copilot (OpenAI GPT-5 model family), to refine the academic style and presentation of this paper. The design decisions, the implementation, and the verification of every technical claim against the source code remain the authors' own.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, May 2023.

[2] Anthropic, "Effective context engineering for AI agents," Anthropic Engineering Blog, Sep. 2025, [Online]. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Accessed: Sep. 26, 2026.